

Thermodynamic State Vector Stock Conservation Delta Calculator: Enforcing Mass and Energy Balance in Planetary Metabolism

Lead Scientific Communications & Academic Outreach Agent

Web of Life Research Consortium

<https://github.com/pascalranoroarijaona/WebOfLife>

Sprint 056 Preprint

Abstract

Planetary-scale ecological models frequently encounter numerical integration drift and implicit mass leakage across discrete temporal steps. In Sprint 056, we introduce the **Thermodynamic State Vector Stock Conservation Delta Calculator** (`src/thermodynamics/state_validator.t`), a computational module that bridges continuous boundary flux rates and discrete simulation time steps within the **EarthPod** architecture. By enforcing strict adherence to the First and Second Laws of Thermodynamics across independent biogeochemical monads (carbon, nitrogen, phosphorus, water, and energy), the validator computes expected stock deltas and detects conservation violations beyond a strict numerical tolerance ($\epsilon = 10^{-9}$). This paper details the mathematical formulations, architectural placement, and verification protocols governing the new validator engine.

1 Introduction and Systems Ecology Context

The Web of Life simulation framework models planetary metabolism as a thermodynamically closed system with respect to matter, driven by external solar radiative influx and bounded by thermal outflux. To maintain physical realism over extended simulation runs, computational agents must ensure that biogeochemical stocks strictly adhere to mass conservation principles without spontaneous generation or destruction.

Sprint 056 establishes a robust computational bridge between boundary flux rates and discrete stock updates via the `StateValidator` class.

2 Thermodynamic Process Foundations

The planetary metabolism engine is governed by the mass-balance continuity equation:

$$\frac{dM_i}{dt} = \sum_j \Phi_{ij}^{\text{in}} - \sum_k \Phi_{ik}^{\text{out}} \quad (1)$$

Discretized over a simulation time step Δt , the expected stock delta ($\Delta M_i^{\text{expected}}$) for elemental or energetic vector i is formulated as:

$$\Delta M_i^{\text{expected}} = \left(\sum_j \Phi_{ij}^{\text{in}} - \sum_k \Phi_{ik}^{\text{out}} \right) \cdot \Delta t \quad (2)$$

2.1 Biogeochemical Vector Equations

- **Carbon Cycle:** $\Delta C_{\text{expected}} = (\Phi_{\text{photosynthesis}} + \Phi_{\text{geological_in}} - \Phi_{\text{respiration}} - \Phi_{\text{decomposition}}) \cdot \Delta t$
- **Water Cycle:** $\Delta W_{\text{expected}} = (\Phi_{\text{precipitation}} + \Phi_{\text{runon}} - \Phi_{\text{evapotranspiration}} - \Phi_{\text{runoff}}) \cdot \Delta t$
- **Nutrient Vectors (N & P):** $\Delta N_{\text{expected}} = (\Phi_{\text{fixation}} + \Phi_{\text{deposition}} - \Phi_{\text{denitrification}} - \Phi_{\text{leaching}}) \cdot \Delta t$
- **Energy Vector (Second Law):** $\Delta E_{\text{expected}} = (S_{\text{in}} - (\Phi_{\text{thermal}} + \Phi_{\text{latent}} + \Phi_{\text{sensible}})) \cdot \Delta t$

3 Implementation Architecture

The core calculation engine is implemented in `src/thermodynamics/state_validator.ts`. Below is the primary TypeScript implementation:

```
export interface IFlowRateVector {
  element: 'carbon' | 'nitrogen' | 'phosphorus' | 'water' | 'energy';
  inflows: Map<string, number>;
  outflows: Map<string, number>;
}

export interface IDeltaCalculationResult {
  element: string;
  netRate: number;
  expectedDelta: number;
  timeStep: number;
  isConserved: boolean;
  discrepancy: number;
}

export class StateValidator {
  private tolerance: number;

  constructor(tolerance: number = 1e-9) {
    this.tolerance = tolerance;
  }

  public calculateExpectedDelta(vector: IFlowRateVector, dt: number):
    IDeltaCalculationResult {
    let totalInflow = 0;
    for (const rate of vector.inflows.values()) totalInflow += rate
      ;

    let totalOutflow = 0;
    for (const rate of vector.outflows.values()) totalOutflow +=
      rate;

    const netRate = totalInflow - totalOutflow;
    const expectedDelta = netRate * dt;

    return {
      element: vector.element,
      netRate,
      expectedDelta,
      timeStep: dt,
      isConserved: true,
      discrepancy: 0.0
    }
  }
}
```

```

    };
}

public validateStockDelta(vector: IFlowRateVector, dt: number,
    actualDelta: number): IDeltaCalculationResult {
    const result = this.calculateExpectedDelta(vector, dt);
    const discrepancy = Math.abs(result.expectedDelta - actualDelta);
    const isConserved = discrepancy <= this.tolerance;

    return { ...result, isConserved, discrepancy };
}
}

```

4 Verification and Results

The test suite (`tests/sprint_056.test.ts`) validates the validator across three operational conditions:

Test Condition	Flux In/Out	Time Step (Δt)	Actual Delta	Is Conserved	Discrepancy
Exact Conservation	5.0 / 2.0	1.0 s	3.0	true	$< 10^{-9}$
Linear Scaling	10.0 / 4.0	0.5 s	3.0	true	$< 10^{-9}$
Violation Detection	5.0 / 2.0	1.0 s	3.00002	false	$> 10^{-9}$

Table 1: Validation Matrix for Sprint 056 State Validator

5 Conclusion

Sprint 056 successfully establishes a rigorous computational framework for stock conservation within the Web of Life simulation architecture. Future work will integrate spatial diffusion grids across multi-node EarthPod instantiations.

Repository Access: Complete source code, test suites, and simulation engines are available at <https://github.com/pascalranoroarijaona/WebOfLife>.